

Drift Assignment — Requirements Document

Project: Android drifting / driving game inspired by *Arabic Drifting* **Engine:** Unity 6000.0.69f1 (URP)
Platform: Android (primary), Editor Play Mode (dev) **Status:** v1.0 — Pre-implementation **Owner:**
Kamlesh (social@tevaeralabs.com)

1. Overview & Goals

A polished, mobile-first driving game inspired by the *Arabic Drifting* Android game. The player drives a realistic-looking low-poly sedan on a long, wide desert road with touch-based controls. The car supports runtime physics tuning (drivetrain, suspension, camber, helper assists), driving-style presets, and a two-stage damage system where collisions first dent panels and, at higher impact, break parts off the vehicle.

Success criteria

- Drivable car with responsive, drift-capable physics on touch input
- Visible, real-time effect from all tuning sliders and presets
- Damage system produces both **dents** (light collisions) and **detached parts** (heavy collisions)
- Assets are 100% free / CC-licensed with attribution
- Codebase follows Unity's official C# style guide (see §9)
- Ships as a signed Android APK

Non-goals (out of scope)

- Multiplayer / networking
 - Save / load / progression system
 - Multiple maps or multiple vehicles at launch
 - IAP, ads, analytics
 - iOS build
-

2. Assumptions

- Single-player, no networking, no persistence across sessions
 - One car and one map at launch — **desert circuit / drift loop** (~600–1000 m lap with straights, hairpins, and a sweeper), styled after the reference desert environment
 - Touch controls on device; keyboard fallback available in the Editor for development
 - **Free / CC0 assets only** — no paid Asset Store purchases
 - Interview evaluators judge on: physics feel, damage system quality, code architecture, mobile performance, and documentation quality
 - Character enter / exit is an **optional stretch** goal — not on the critical path
 - Reference gameplay: *Arabic Drifting* mobile game (screenshots embedded below)
-

3. Technical Stack

Layer	Choice	Justification
Engine	Unity 6000.0.69f1	Already installed, LTS-track
Render pipeline	URP 17.0.4	Mobile-optimized, already installed
Physics	Built-in PhysX + WheelCollider	Battle-tested, no license issues
Input	Unity Input System 1.19.0	On-screen touch + Editor keyboard fallback
Camera	Cinemachine (to be added)	Chase, hood, cinematic, look-back cams
Animation (stretch)	Mixamo rigs + Animator	Free, standard for character work
Build	IL2CPP, ARM64, .NET Standard 2.1	Google Play requirement (64-bit)
Editor tooling	Unity MCP (CoplayDev)	Agentic Editor automation — scene edits, prefab wiring, tests

Unity MCP registration (local scope, HTTP transport):

```
claude mcp add --scope local --transport http UnityMCP http://127.0.0.1:8080/mcp
```

Cleanup:

```
claude mcp remove --scope local UnityMCP
claude mcp remove --scope user UnityMCP
claude mcp remove --scope project UnityMCP
```

List active servers: `claude mcp list`

Unity Editor must be running with the CoplayDev Unity MCP window open (serving on 127.0.0.1:8080/mcp) before Claude can invoke Editor tools.

4. Gameplay Features (extracted from reference screenshots)

4.1 HUD (in-game screen)

- Analog **RPM tachometer** with needle
- Current **gear** display
- **Speed** readout (KM/H)
- **Headlights** toggle
- On-screen **steering** (left / right arrow buttons, analog via drag)
- **Gas + brake** pedal buttons (press & hold)
- **Handbrake** button
- **Camera cycle** button
- Recording toggle + replay controls (*stretch*)

- Photo-mode button (*stretch*)
- Camera-angle switcher (*stretch*)

4.2 Settings Menu

- Transmission: **Automatic** ↔ **Manual**
- Recording on / off (*stretch*)
- **Hydraulics** on / off (*stretch* — *bounce* / *lift effect*)
- Buttons: **Continue**, **Restart**, **Return to Garage**, **Exit**
- Sub-menus: Advanced Tuning, Hydraulics, Weight Distribution, Control Type

4.3 Advanced Tuning

- **Front Suspension Spring Force** slider (visual tint changes with value)
- **Rear Suspension Spring Force** slider
- **Helper Value** slider (assistive handling — traction / steering assist)
- **Drivetrain** dropdown: **FWD** / **RWD** / **AWD**

4.4 Weight & Camber

- **Front / Rear Suspension** height sliders
- **Front / Rear Camber** angle sliders

4.5 Preset Picker

- Numbered presets **0–13** with named driving styles ("Sa Style", "Heavy — Pros Only", "Hujuli w Khabat", etc.)
- Confirm button applies the preset's values to all tuning sliders

5. Driving Physics Spec

- **Model**: WheelCollider-based, sim-cade tuning (arcade-drifty feel, not sim)
- **Center of mass**: manually lowered via `Rigidbody.centerOfMass` offset to prevent rollover
- **Drivetrain modes** (`DrivetrainMode { Fwd, Rwd, Awd }`): torque distribution changes with mode
- **Drift enablers**:
 - Reduced rear lateral stiffness on handbrake
 - Forward stiffness dynamic with speed
- **Gearing**: 6-speed **auto** + **manual** mode; RPM-driven shift points
- **Steering**: speed-sensitive steering angle (more angle at low speed, less at high)
- **Handbrake**: cuts rear brake torque and drops rear stiffness for controlled slides
- **Helper Value**: `0` = raw physics, `1` = full traction / counter-steer assist (linear blend)

6. Damage System Spec

Hero car is **RMCar26** (from Realistic Mobile Cars — Pro3DModels demo pack, see §8). The prefab is already articulated into separate GameObjects per panel, door, glass pane, and wheel — no manual splitting

needed.

Both stages driven by per-part `accumulatedImpact` (running sum of impulse magnitudes above a small floor threshold).

Stage A — Dent (0 → dent threshold)

- Real-time mesh vertex displacement in a radius around each contact point
- Vertices displaced along surface normal, weighted by distance falloff
- Original vertex positions cached on `Awake` for reset / debug
- MeshCollider (convex) stays static; only the visual mesh deforms — keeps physics stable and cheap
- **Applied to (RMCAR26 mesh names):**
 - `Body_A` — main body shell
 - `BodyFrontBumper_A` , `BodyFrontBumper2_A` — front bumper (dents before detachment)
 - `BodyRearBumper_A` — rear bumper
 - `Body_A_DoorFrontLeft` , `Body_A_DoorFrontRight` — front-door skin panels
 - `Body_A_DoorRearLeft` , `Body_A_DoorRearRight` — rear-door skin panels

Stage B — Break (dent threshold → break threshold)

- Each detachable part starts parented to the car; on threshold hit: unparent, add `Rigidbody` , add convex `MeshCollider` , inherit car's velocity, add impact impulse
- Broken parts live for **N seconds** then despawn (mobile memory budget)
- **Detachable parts (RMCAR26 mesh names):**
 - `BodyFrontBumper_A` + `PaintFrontBumper_A` — front bumper (skin + paint layer detach together)
 - `BodyRearBumper_A` — rear bumper
 - `DoorFrontLeft` , `DoorFrontRight` — front doors
 - `DoorRearLeft` , `DoorRearRight` — rear doors
 - `WheelFrontLeft/Right` , `WheelRearLeft/Right` — wheels at extreme damage only
 - `LicensePlate` , `PaintSpoiler_A` — cosmetic bits shed early
- **Glass shatter** (visual swap, not physical detach):
 - `GlassFront` (windshield): shader swap to shattered glass on heavy front impact
 - `GlassRear` : same on rear impact
 - `WindowsFrontLeft/Right` , `WindowsRearLeft/Right` , `Windows` (side panels): hide + spawn shard particle burst on side impact

Character enter/exit tie-in

The four `Door*` GameObjects are individual meshes with their own pivots. **Character enter/exit** (§7 stretch goal) hinge-rotates `DoorFrontLeft` (or `DoorFrontRight`) around its Y-axis for open/close animation before hiding the character mesh into the car interior. No additional rigging needed.

Dashboard tie-in

The prefab exposes `SteeringWheel` , `rpmArrow` , `SpeedArrow` as separate GameObjects — the in-car HUD can drive their local rotation directly from `CarController` events, giving a working analog dashboard without extra art.

Fallback path

If runtime mesh deformation proves too costly on target Android hardware, swap to a **3-tier prefab-swap** approach: `pristine` → `dented mesh variant` → `broken/parts-hidden variant` . This fallback is toggleable per platform; documented explicitly rather than silently discovered later.

7. Optional — Character Enter / Exit (Stretch)

- Third-person character rigged from Mixamo
- Interaction trigger volume near driver door
- **On enter**: character animates open-door → sit → hide mesh; camera swaps to chase cam
- **On exit**: reverse; camera swaps to third-person follow

Only shipped if polished; a half-finished stretch feature hurts more than skipping it.

8. Free Asset Sources

Style target: low-poly but *slightly realistic* — clean silhouettes, mature color palette, no thick cartoon outlines. Kenney's aesthetic (cartoony, chunky, kid-game vibe) is **not** the target. We prefer **Quaternius**, **Sketchfab CC0**, and **Poly Pizza** for meshes.

Asset	Source	License	Notes
Hero car (chosen)	Realistic Mobile Car #26 (Demo) — Surdov Vadym	Unity Asset Store EULA (free demo)	RMCAR26 — fully articulated: separate bumpers, 4 doors, side/front/rear glass, wheels, brake rotors, lights, working dashboard parts. LODs 0–3 baked in.
Additional cars (later)	Sketchfab — CC0 filter	CC0	Only if we ship the garage car-select — otherwise skip
Circuit / road system	EasyRoads3D Free (Asset Store)	Free	Spline-based circuit builder, terrain-aware
Circuit (alt / DIY)	Unity Splines (already installed) + custom mesh extrusion	—	Fallback if EasyRoads3D free tier limits us
Desert props (dunes, rocks)	Quaternius Ultimate Nature	CC0	Cleaner style than Kenney
Palm trees	Quaternius Ultimate Nature or Poly Pizza — "palm"	CC0	Line the circuit as in reference
Skybox	Poly Haven — desert HDRIs	CC0	Clear-sky HDRIs (2K is enough)
Character rig + anims (stretch)	Mixamo	Free (Adobe account)	Idle, walk, enter-car, exit-car anims
Car physics reference	Prometeo Car Controller	MIT	Reference only — we write our own
Impact / crash SFX	Freesound.org (CC0 filter)	CC0	Crashes, dents, glass breaks
Engine audio	Sonniss GDC packs	Royalty-free	High-quality engine loops
UI icons	Kenney Game Icons / Input Prompts	CC0	Icons only — flat vector, style-neutral, fine to use here
Fonts	Google Fonts (SIL Open Font License)	OFL	Any techy sans-serif

Attribution: every used asset is tracked in [CREDITS.md](#) with source + license.

9. Code Style & Conventions (mandatory)

Source of truth: Unity's official C# style guide → <https://unity.com/how-to/naming-and-code-style-tips-c-scripting-unity>

Key rules (excerpt — full guide governs any ambiguity):

- **Classes / methods / properties / enums / enum members:** `PascalCase`
- **Local variables & parameters:** `camelCase` (`impactForce` , `wheelIndex`)
- **Private fields:** `_camelCase` with leading underscore (`_rigidbody` , `_maxTorque`)

- **Public fields** (avoid — prefer properties): `PascalCase`
- **Constants**: `PascalCase` (not `SCREAMING_SNAKE_CASE`)
- **Interfaces**: `IPascalCase` (`IDamageable` , `IInputProvider`)
- **Enums**: `PascalCase` type name; `PascalCase` members; **singular** name unless `[Flags]` (then plural).
Example:

```
public enum DrivetrainMode { Fwd, Rwd, Awd }

[Flags]
public enum DamageStages { None = 0, Denting = 1, Breaking = 2 }
```

- **Braces**: Allman style (opening brace on its own line)
- **Namespaces**: `PascalCase` , one namespace per file, mirrors folder path (e.g. `DriftAssignment.Vehicle`)
- **Async methods**: suffix with `Async`
- **Serialized private fields**: `[SerializeField]` `private float _maxTorque;` — expose via property or method, not public field
- **Abbreviations**: avoid except widely accepted (`Id` , `Ui` , `Url`)
- **File name matches primary type name**

Enforcement: an `.editorconfig` at repo root mirrors these rules so Rider / VS auto-formats on save.

10. Project Structure (target)

```
Assets/
  _Project/
    Scripts/
      Vehicle/      CarController, WheelController, GearBox, HandBrake, Drivetrain
      Damage/       ImpactReceiver, DentableMesh, DetachablePart, DamageController
      Input/        IInputProvider, TouchInputBridge, KeyboardInputProvider
      UI/           HudController, TuningMenuController, PresetPicker
      Camera/       CameraRig (Cinemachine wrappers)
      Character/    CharacterEnterExit [stretch]
    Prefabs/
      Vehicles/     Car_Sedan.prefab, wheel prefabs, part prefabs
      Environment/  RoadSegment, Dune, PalmCluster, Lamppost
      UI/           HUD.prefab, TuningMenu.prefab
    ScriptableObjects/
      TuningPresets/ Preset_00..Preset_13 assets
      CarConfig/    Baseline vehicle stats
      Scenes/       Main.unity, Garage.unity [stretch]
      Materials/, Textures/, Audio/, Fonts/
    ThirdParty/
      Kenney/, Quaternius/, Mixamo/ (each with LICENSE.txt)
  Doc/              Requirements, Architecture, Implementation log, Optimization log, Credits,
  PDFs
```

Assembly Definitions: `DriftAssignment.Core` , `.Vehicle` , `.Damage` , `.Input` , `.UI` , `.Camera` .

11. Implementation Phases

Each phase below produces observable output. The [Implementation Log](#) captures the actual work done, screenshots, and any dead ends per phase.

#	Phase	Output
0	Initial Project Setup	Android target, URP tuned, folder tree, asmdefs, MCP verified
1	Environment & Assets	Desert circuit scene (~600–1000 m loop) with straights, hairpins, sweeper
2	Car Physics	Drivable sedan (keyboard in Editor)
3	Camera System	Chase / hood / cinematic / look-back Cinemachine cams
4	Touch HUD	On-screen controls: steering, pedals, gear, RPM, camera
5	Tuning Menu	Sliders wired to live physics
6	Preset System	14 preset SOs + picker UI
7	Damage System	Dent stage + break stage
8	Audio / Particles / Polish	Engine loop, tire smoke, skid marks, post-processing
9	Performance Optimization	Cross-cutting; see OPTIMIZATION.md
10	Character Enter / Exit (<i>stretch</i>)	Third-person walk → enter → drive → exit loop
11	Documentation Finalization	Docs finalized + PDF export

12. Verification — how "done" is measured

- Requirements doc reviewed and signed off before any Unity work
 - **Physics**: car accelerates, brakes, steers, drifts on handbrake, respects the selected drivetrain
 - **Damage**: light collision → visible dent; heavy collision → part detaches
 - **Presets**: switching preset 0 → 13 produces visibly different handling
 - **Android APK** builds cleanly and runs on-device
 - **Assets**: every asset attributed in `CREDITS.md` with license
 - **Docs**: `REQUIREMENTS.md` , `ARCHITECTURE.md` , `IMPLEMENTATION_LOG.md` , `OPTIMIZATION.md` , `CREDITS.md` all present and PDF-exported
-

13. Reference Screenshots

Screenshots from the reference *Arabic Drifting* gameplay are stored in `Doc/reference/` and referenced by section 4. Add screenshot images to that folder before final PDF export.

Suggested filenames:

- `reference/hud.png`
- `reference/settings.png`
- `reference/tuning_springs.png`

- `reference/tuning_drivetrain.png`
- `reference/tuning_camber.png`
- `reference/presets.png`